# Lecture 2

# Welcome to Data Structures
# Spring 2026

במקרה אזעקה:

- אם האזעקה אצל המרצה – השיעור יופסק עד חזרה לשגרה
- אם האזעקה רק אצל הסטודנטים:
  - סטודנט יסמן "הרמת יד" וילך להתמגן
  - במידה ויותר מ-30% מהסטודנטים מתמגנים – השיעור יופסק עד לחזרה לשגרה.

- בכל מקרה, השאירו את ה-zoom פתוח עד החזרה – כדי לחסוך זמן התחברות חוזר.

- גם במידה ומשך ההתגוננות עובר את מועד סוף השיעור – ניפגש ל-2 דקות לסגור את השיעור בצורה מסודרת ולהודעות.

# Welcome to Data Structures Spring 2026

**In Case of Siren**

- If the siren sounds at the instructor's location – the class will be paused until the situation returns to normal.
- If the siren sounds only for the students:
  - A student should use the "Raise Hand" feature and go to a protected area.
  - If more than 30% of the students take shelter – the class will be paused until the situation returns to normal.

- Please keep the Zoom session open until we return, to avoid reconnection delays.
- Even if the sheltering period extends beyond the scheduled end of the class, we will reconnect for two minutes to close the session properly and share any final announcements.

# Previous Lecture

- Terminology:
  - Algorithm
  - Abstract Data Type (ADT)
  - Logical Data Structure
  - Implementation
- Selecting the Data Structure
  - Match the problem in hand
  - Allows the best match for Algorithm efficiency (of many kinds)
- Sequence
  - ADT
  - Implementations
    - Array => stack
    - Circular Array => queue + deque
    - Singly Linked-List => Concatenation + Stack (no Queue)

# Choice of Data Structures & Algorithm affects

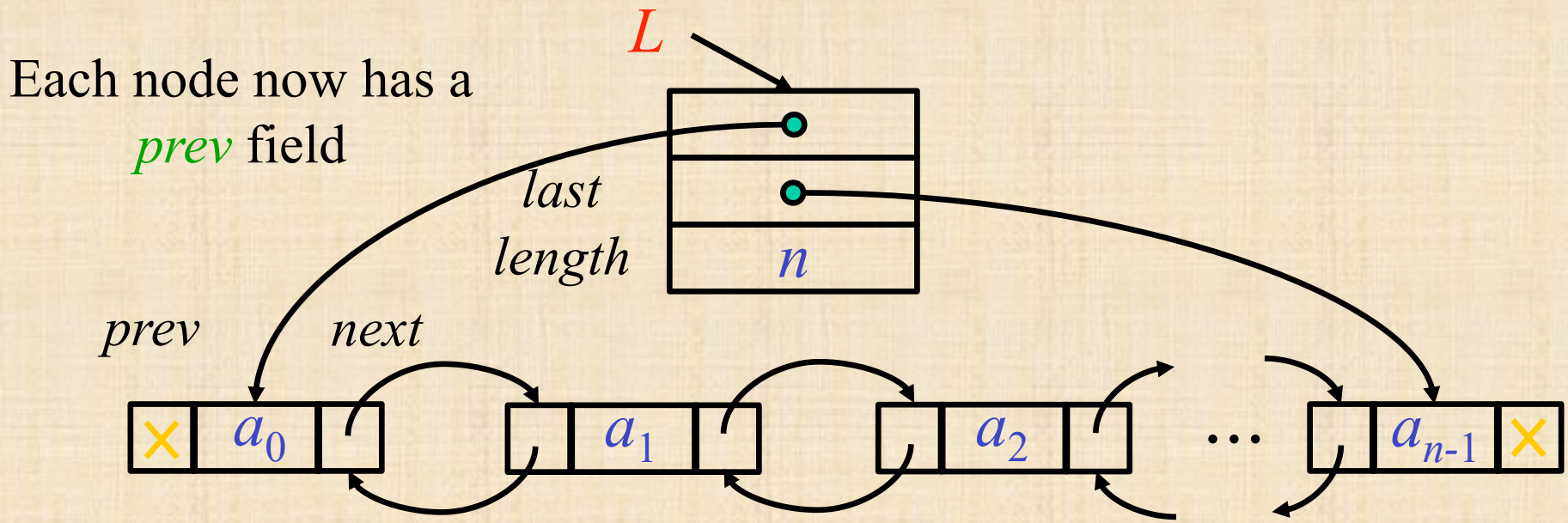
- Time efficiency
- Memory (storage) efficiency
- Ease of Implementation
- Ease of Debugging

All depend crucially on how data is structured

# Implementing Sequence using Doubly Linked List

Insert/Delete-Last in O(1) time

Concatenation also in O(1) time

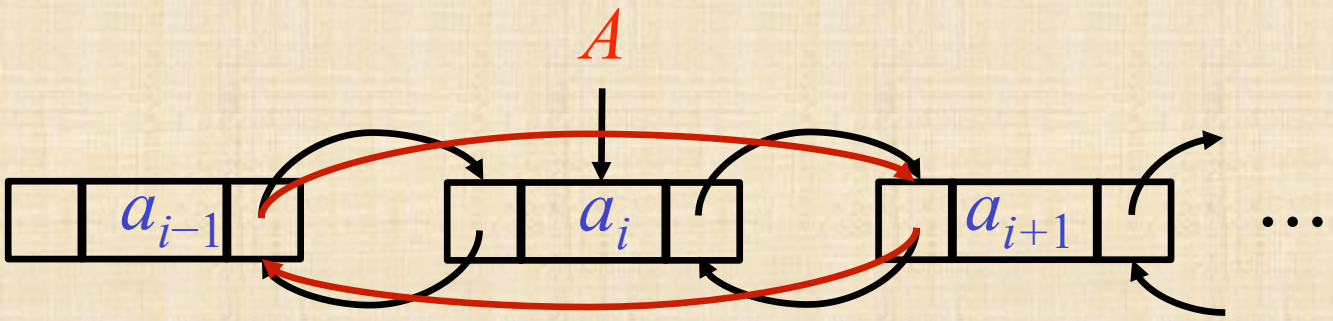



Careful, first and last elements are special.

# Deleting a node from a Doubly Linked List

Each node now has a *prev* field

Delete($A$) – Delete node $A$ from its list




**Function** `Delete(A)`

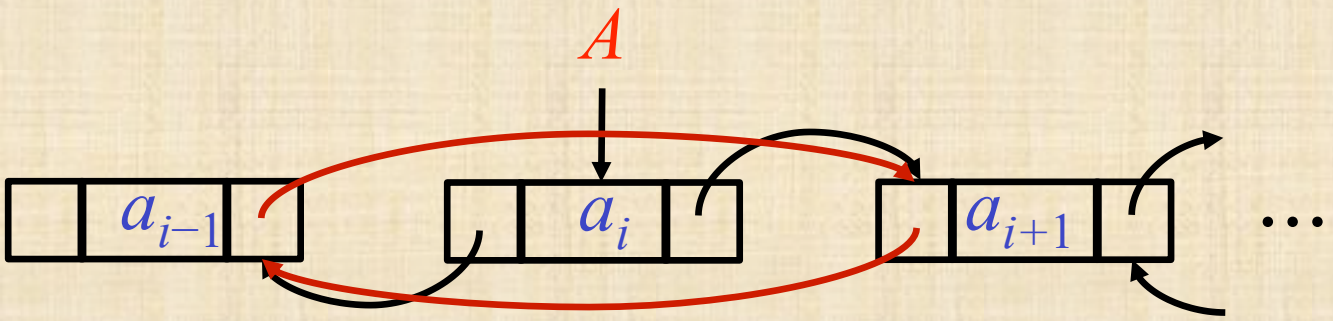
$A.prev.next \leftarrow A.next$
$A.next.prev \leftarrow A.prev$

Note: $A$ itself not changed!

# Deleting a node from a Doubly Linked List

Each node now has a *prev* field

Delete($A$) – Delete node $A$ from its list




**Function** `Delete(A)`

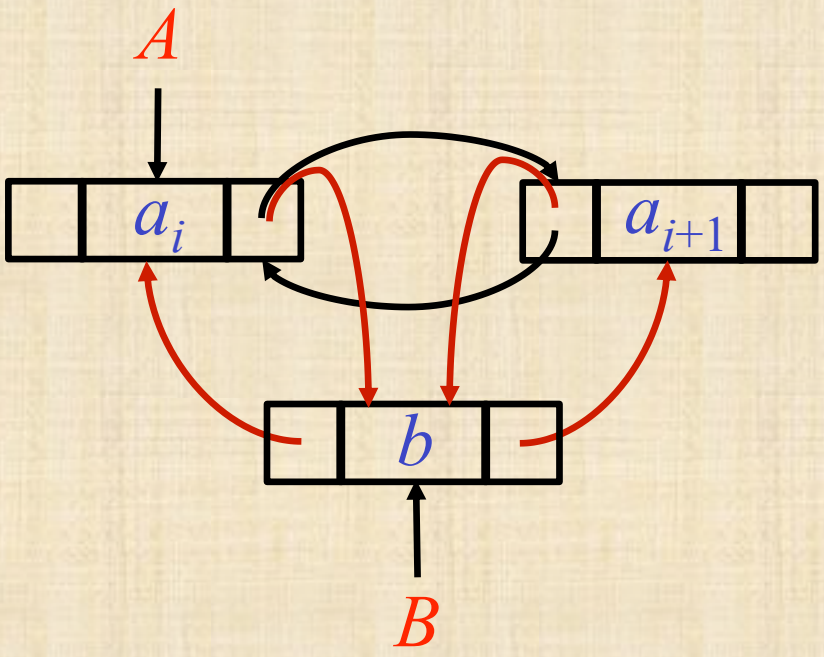
$A.prev.next \leftarrow A.next$
$A.next.prev \leftarrow A.prev$

Note: $A$ itself not changed!

We also did not take care of L.Length

# Inserting a node into a Doubly Linked List

Insert-After($A$,$B$) – Insert node $B$ after node $A$




**Function**
`Insert-After(`$A, B$`)`

$B.prev \leftarrow A$
$B.next \leftarrow A.next$

$A.next \leftarrow B$
$B.next.prev \leftarrow B$

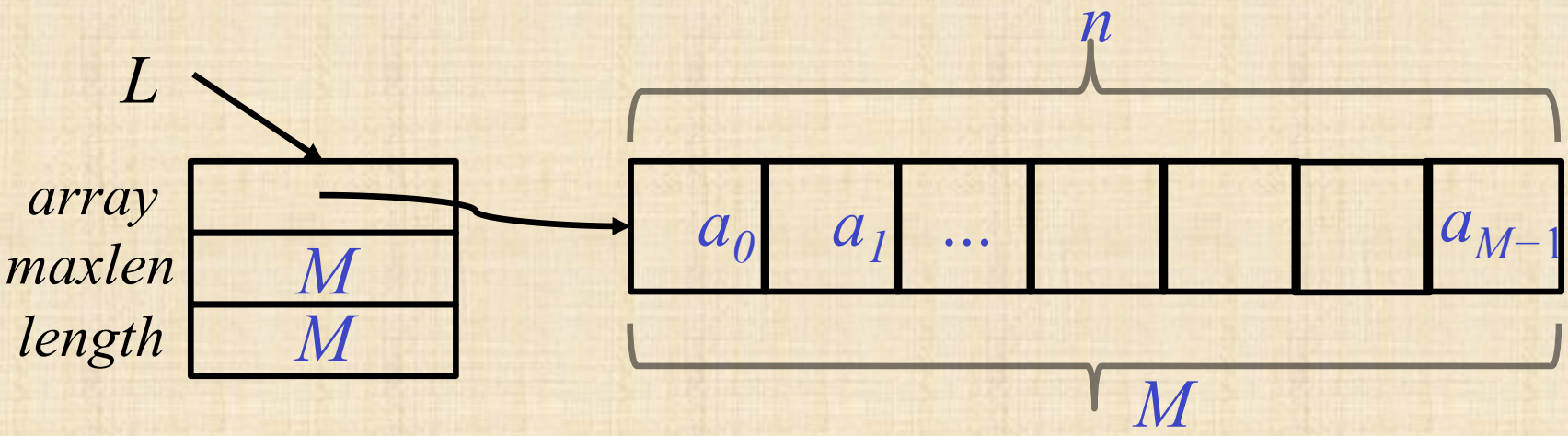
# Circular arrays vs. Linked Lists

| | Circular arrays | Doubly Linked lists |
|---|---|---|
| Insert/Delete-First | $O(1)$ | $O(1)$ |
| Insert/Delete-Last | $O(1)$ | $O(1)$ |
| Insert/Delete($i$) | $O(\min\{i+1, n-i+1\})$ | $O(\min\{i+1, n-i+1\})$ |
| Retrieve($i$) | $O(1)$ | $O(\min\{i+1, n-i+1\})$ |
| Concatenation | $O(\min\{n_1, n_2\}+1)$ | $O(1)$ |

# Question

- Problem definition:
  - Need direct access
  - Don’t know what is the maximal size

# Implementing lists using (circular) arrays with resizing



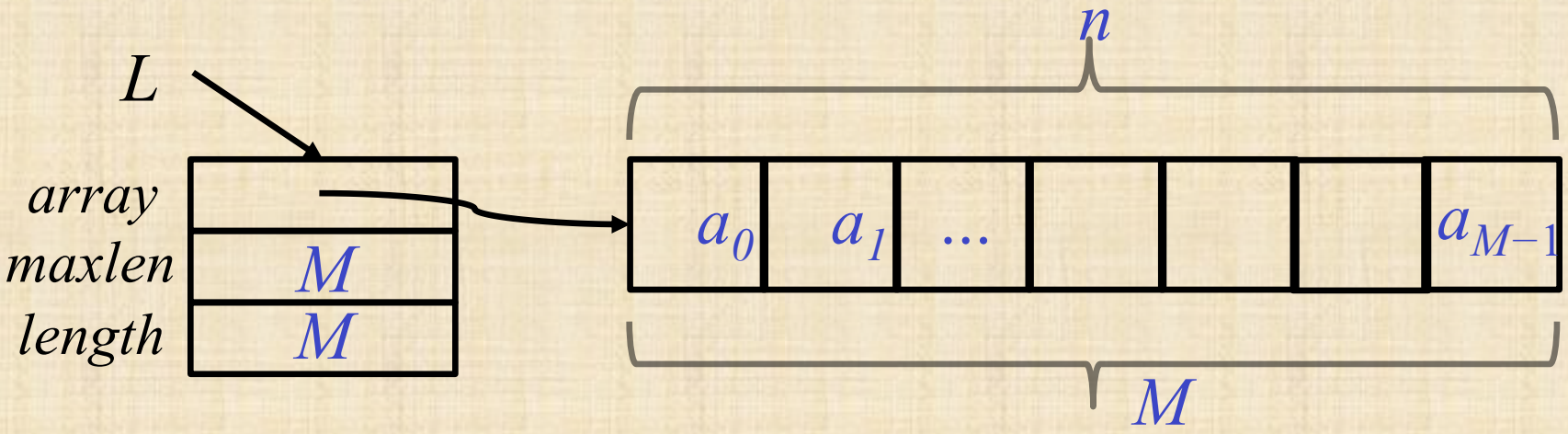

What do we do when the array is full?

We cannot extend the array.

Allocate a larger array and copy

What should be the size of the new array?

# Implementing lists using (circular) arrays with resizing



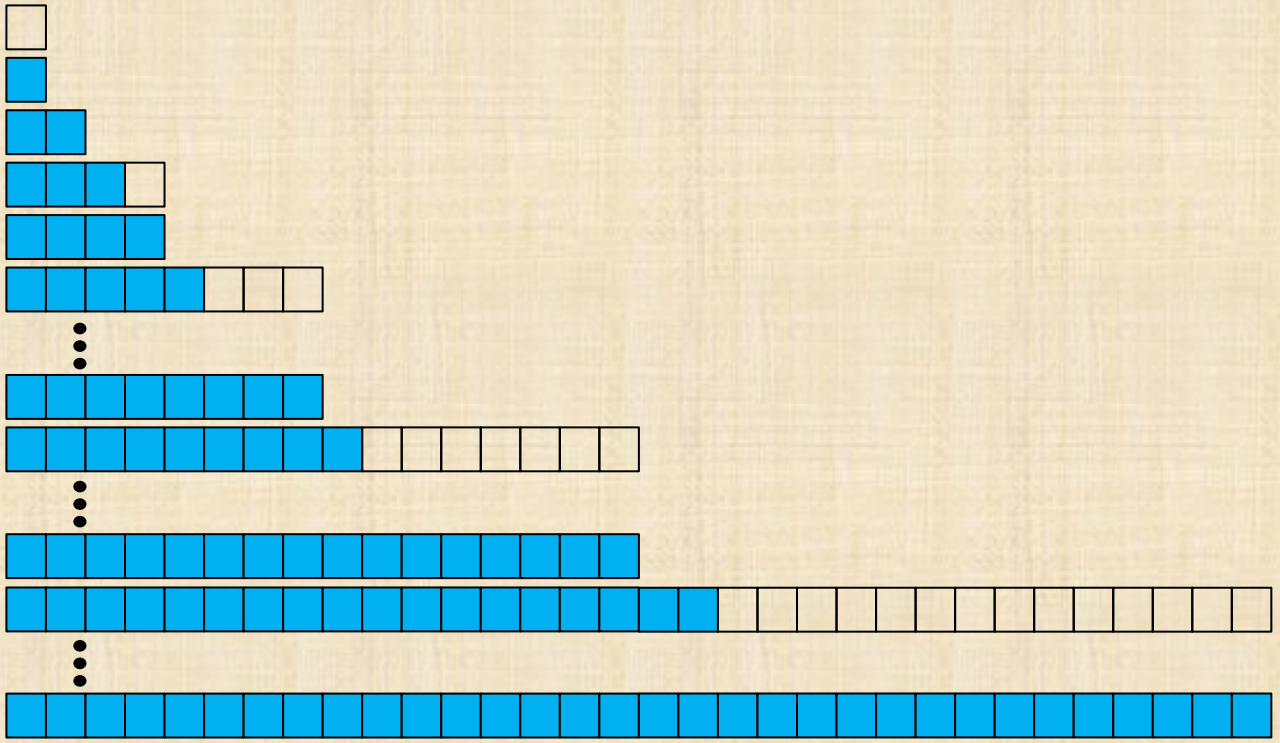

If we start with an empty array and increase its length by 1, then the time to do $n$ Insert-Last is about:

$$1 + 2 + \ldots + n = \frac{n(n+1)}{2} = O(n^2)$$

# Implementing lists using (circular) arrays with doubling

When the array is full, double its size and copy.

What is the cost of $n$ Insert-Last operations?



| inserts | copies |
|---|---|
| 0 | 0 |
| 1 | 0 |
| 1 | 1 |
| 1 | 2 |
| 1 | 0 |
| 1 | 4 |
| ⋮ | ⋮ |
| 1 | 0 |
| 1 | 8 |
| ⋮ | ⋮ |
| 1 | 0 |
| 1 | 16 |
| ⋮ | ⋮ |
| 1 | 0 |
| n | 1+2+4+..+n/2 |

# Implementing lists using (circular) arrays with doubling

When the array is full, double its size and copy.
What is the cost of $n$ Insert-Last operations?
Assume, for simplicity that $n=2^k$

$$\underbrace{(1+1+1+\ldots+1)}_{\text{cost of n insertion}} + \underbrace{(1+2+4+\ldots+\frac{n}{2})}_{\text{cost of copying}} = n + (n-1) = O(n)$$

We say that The amortized cost of each operation is O(1)

# Worst-case bounds

*worst(op)* = maximum time for the operation *op*.

The time to serve a sequence of n operations:

$$Time(n \times op) \leq n \times worst(op)$$

Sometimes, this bound is very loose:

$$worst(\texttt{Insert-Last}) = O(n)$$

$$n \times worst(\texttt{Insert-Last}) = O(n^2)$$

However:

$$Time(n \times \texttt{Insert-Last}) = O(n)$$

# Amortized analysis

We say that *amort*(*op*) is an amortized bound on the cost of operation *op* iff for every *n* and every sequence of *n* operations *op*

$$amort(op) \geq Time(n \times op) \;/\; n$$

$$worst(\texttt{Insert-Last}) = O(n)$$

$$amort(\texttt{Insert-Last}) = O(1)$$

$$Time(n \times \texttt{Insert-Last}) \leq n \cdot amort(\texttt{Insert-Last}) = O(n)$$

# Amortized analysis

**amortized analysis** bounds the average running time per operation over the worst-case sequence of operations.

Amortized analysis is useful when what we really care about is the time of the entire sequence, not the time of each individual operation.

In other cases, such as real-time applications, where we need each individual operation to be fast, we should use worst case analysis.

# Implementing lists using (circular) arrays with doubling

When the array is full, double its size and copy.
What is the cost of $n$ Insert-Last operations?
Assume, for simplicity that $n=2^k$

$$\underbrace{(1+1+1+\ldots+1)}_{\text{cost of n insertion}} + \underbrace{(1+2+4+\ldots+\frac{n}{2})}_{\text{cost of copying}} = n + (n-1) = O(n)$$

The amortized cost of each operation is $O(1)$
The worst-case cost is $O(n)$

# Implementing lists using (circular) arrays with doubling

When the array is full, double its size and copy.

$n=2^k$ seems to be a 'fortunate' case.

In general, $n=2^k+r$ for $0 \le r < 2^k$

$$\underbrace{(1+1+1+\ldots+1)}_{\text{cost of n insertion}} + \underbrace{(1+2+4+\ldots+2^k)}_{\text{cost of copying}} = \underbrace{n+(2^{k+1}-1)}_{\le n+2n-1} = \underbrace{O(n)}_{3n-1}$$

The amortized cost of each operation is still $O(1)$

# Amortized Analysis
# ≠
# Average case

- **amortized analysis** bounds the average running time per operation over the worst-case sequence of operations.
- **Average case analysis** bounds the average time of a single operation over all possible inputs

# Prioritizing

- The priority in a (regular) queue is based on arrival time
- There are cases where we would like to define priority differently
- Examples:
  - Broadcast next the most popular song
  - Serve the oldest/youngest costumer

# Priority Queue (ADT)

- ADT: maintain a set of elements, where each element has a priority (key)
- Operations:
  - Insert(x,k): insert element x with key k
  - FindMin(): return a handle referencing an element with a minimum key
  - DeleteMin(): delete an element with minimum key and return a handle referencing it
  - DecreaseKey(h,k): Decrease to k the key of an existing element in the Queue referred to by the handle h (increase the priority of an element)
- Naïve solution?

# Priority Queue

- ADT: maintain a set of elements, where each element has a priority (key).
  - Insert(x,k): insert element x with key k
  - FindMin(): return a handle referencing an element with a minimum key
  - DeleteMin(): delete an element with minimum key and return a handle to referencing it
  - DecreaseKey(h,k): Decrease to k the key of an existing element in the Queue referred to by the handle h
- Naïve solution?
  - Keep a sequence sorted by priority
- There are better solutions! Based on trees

# Recursive Definition of a Tree

A tree is a set of nodes, which is either

- empty

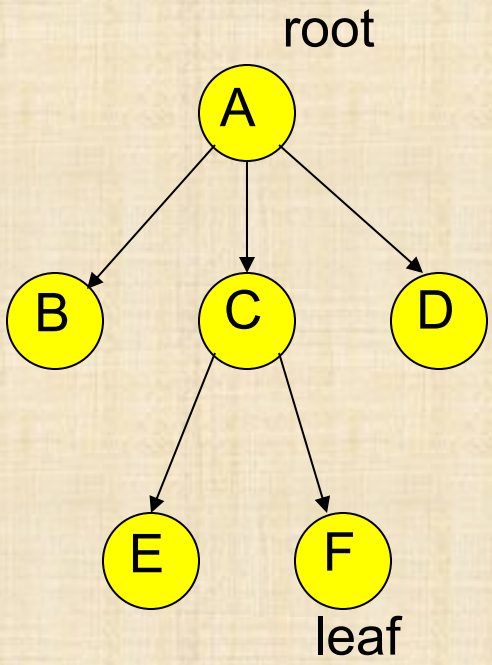
  or

- has one node called the root which points to the roots of zero or more mutually disjoint trees (called subtrees)

  disjoint: no shared nodes

# What is a Tree? (non-recursive)

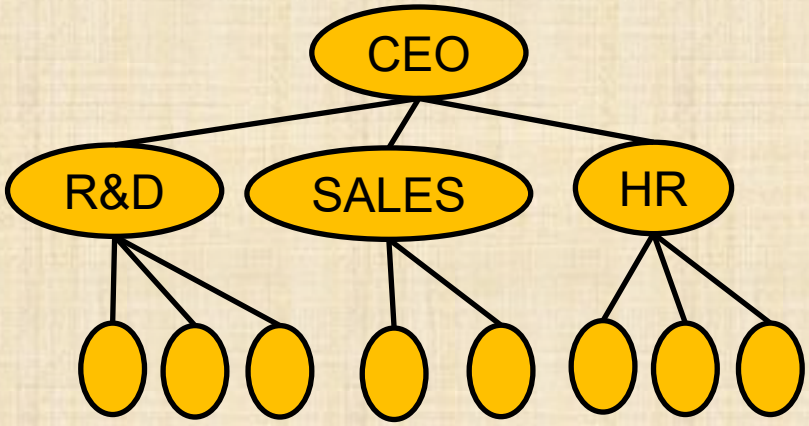
- A structure consisting of levels
- A single element in the top level
- Each element at level *i* points to zero or more elements at level *i+1*
- Each element in level *i+1* is pointed to by a single element in level *I*

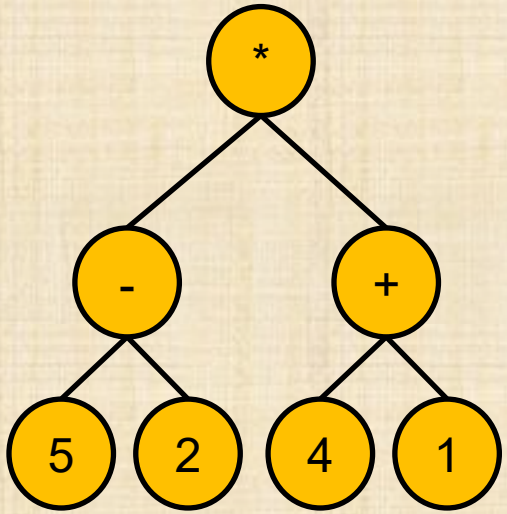



- We call the elements *nodes* (or *vertices*)
- We say that when a node A points to node B they are connected by an *(directed) edge*

# Why Do We Need Trees?

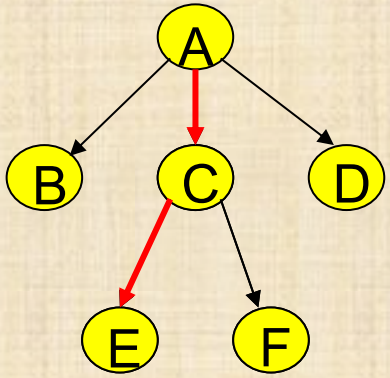
- Lists, Stacks, and Queues are linear relationships
- Information often contains hierarchical relationships
  - File directories or folders
  - Moves in a game
  - Hierarchies in organizations



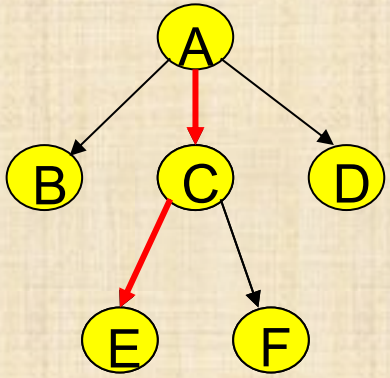

(5-2)*(4+1)

# Tree Terminology




- *Root:* the only node with no parent
- *Child:* B is a child of A if A points to B
- *Parent:* A is a parent of B if A points to B
- *Leaf*: a node with no children
- *Internal node*: any node which is not a leaf
- *Sibling*: A and B are siblings if they have the same parent

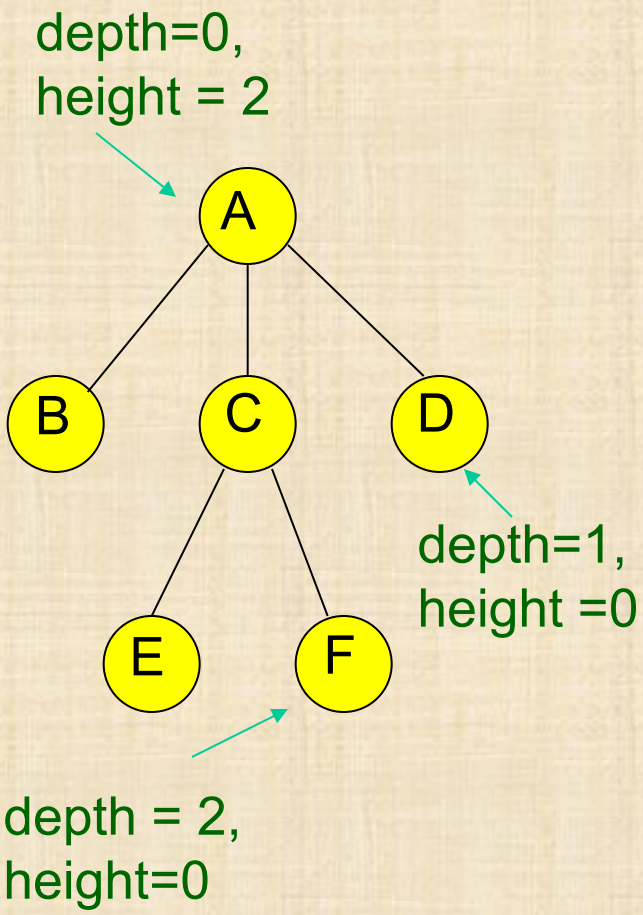
# Tree Terminology




- *Path (directed):* A sequence of nodes where each node is the parent of its successor in the sequence
- *Length of a path:* the number of edges in the path
- *Ancestor:* A is ancestor of B if there is a path starting at A and ending at B
  - *Strict:* The path is of length at least 1
  - *Non-Strict:* The path be of length 0 (empty path)

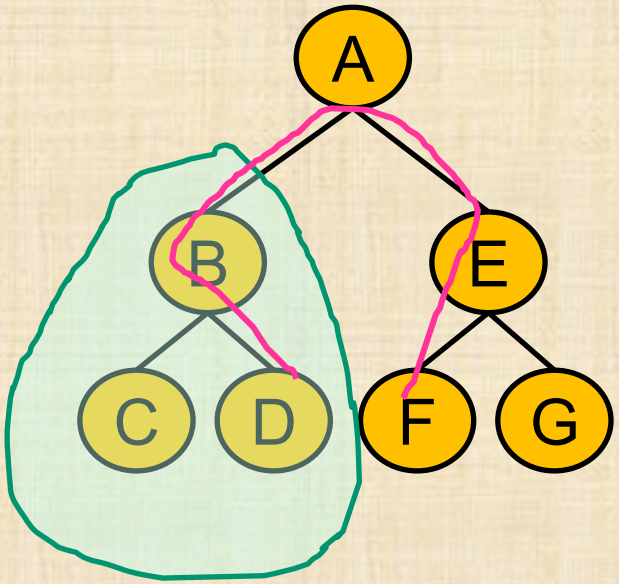
# More Tree Terminology

- *Length of a path:* the number of edges in the path
- *Depth* of a node *N:* the length of path from root to *N*
- *Depth of tree:* the depth of deepest node
- *Height of node N:* the length of longest path from *N* to a leaf
- *Height of tree:* the height of the root (= depth of tree)



# Trees - Terminology

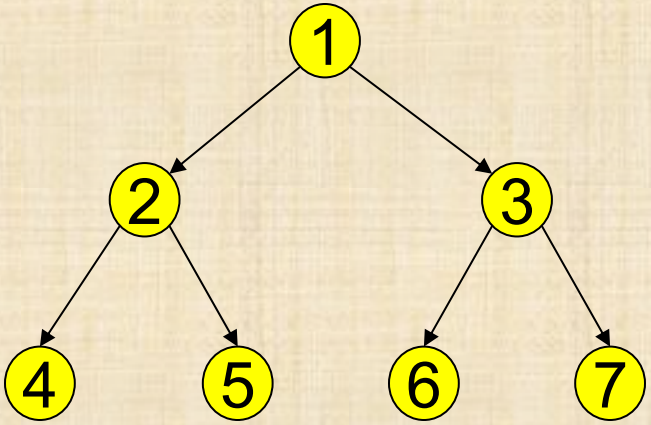
- ordered / unordered trees
  Is there an order defined on the children of each node
- *subtree rooted at B*
- *Path (undirected) between D and F*
- *Degree of a node A:* the number of children of A
- *Binary tree:* every node has degree at most 2.




Degree(B)=2
Degree(F)=0

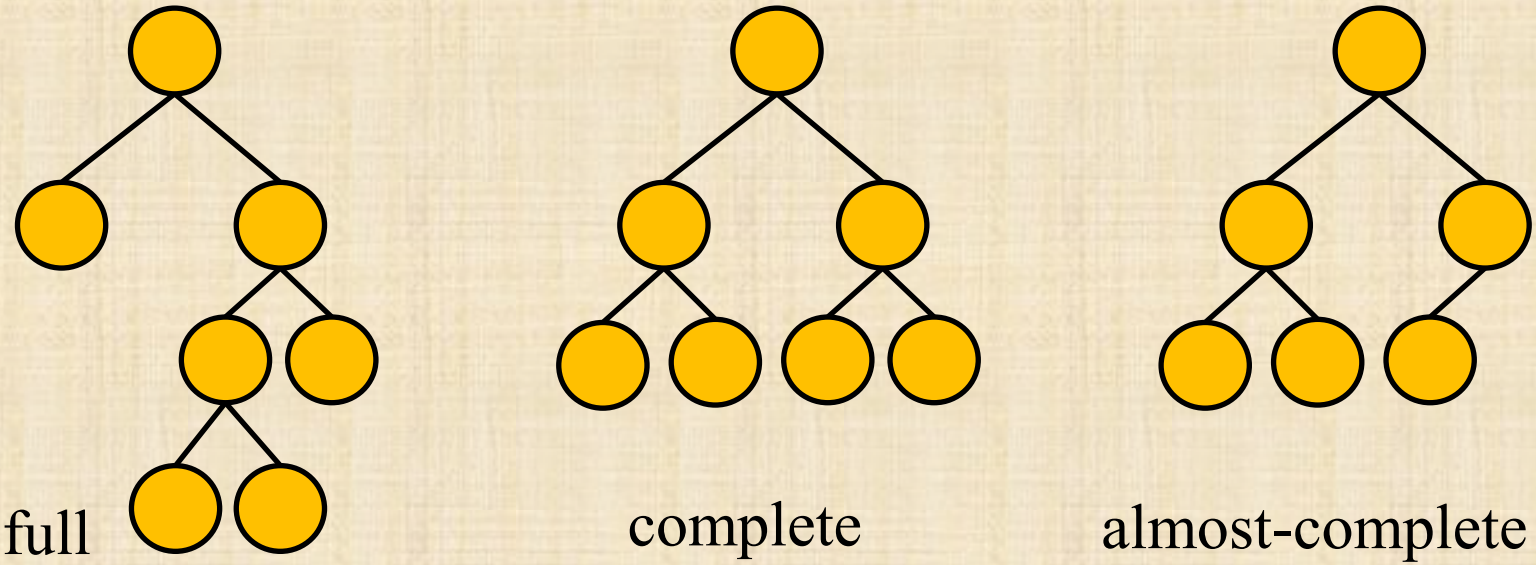
# Binary Trees (BT)

- An ordered tree where each node has at most two children
  - Most popular tree in computer science



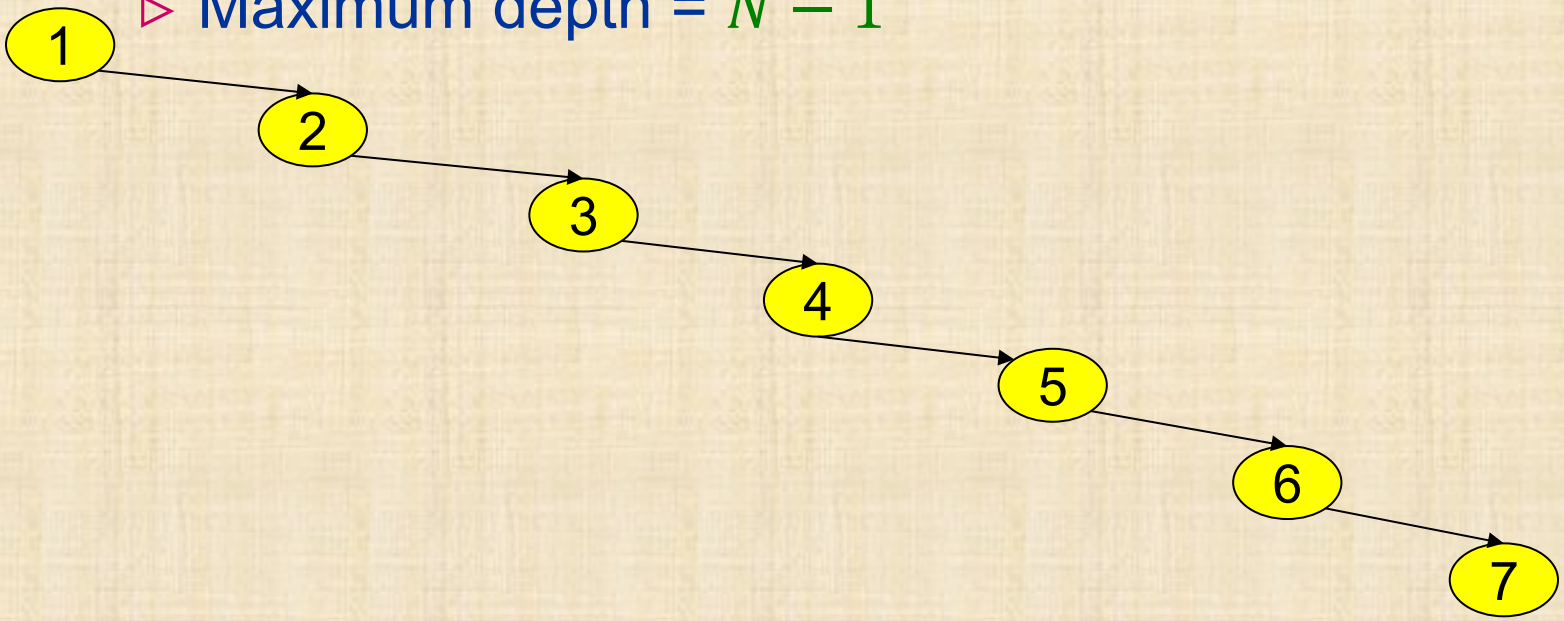
# Binary Trees

- a binary tree is *full* if every node has degree 2 or 0
- a binary tree is *complete* if it is full and all leaves are at the same depth
- an *almost-complete* binary tree might have a few nodes missing at the end of the "deepest level".



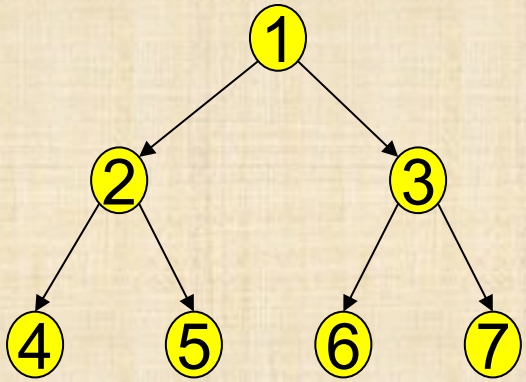
# Maximum Depth given Node Count

- What is the maximum depth of a binary tree with $N$ nodes?
  - Degenerate case: Tree consists of a single path!
  - Maximum depth = $N - 1$



# Maximum Node Count given Depth

- Consider a complete BT of depth $d$:
  - Every internal (non-leaf) node has 2 children
- Number of nodes at each level:
  - Depth 0 (the root): 1
  - Depth 1: 2
  - Depth *k:* $2^k$
- Total: $\sum_{k=0}^{d} 2^k = 2^{d+1} - 1$

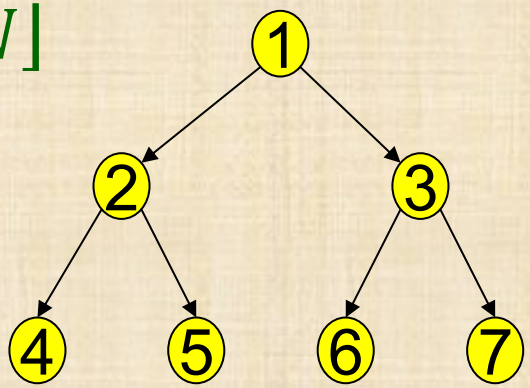


# Minimum Depth given Node Count

- What is the minimum depth of a binary tree, $d_{min}(N)$, with $N$ nodes?

- Obtained for almost complete binary trees: all levels except possibly the last one are full

- We will show $d_{min}(N) = \lfloor \log_2 N \rfloor$



# Calculating $d_{min}(N)$

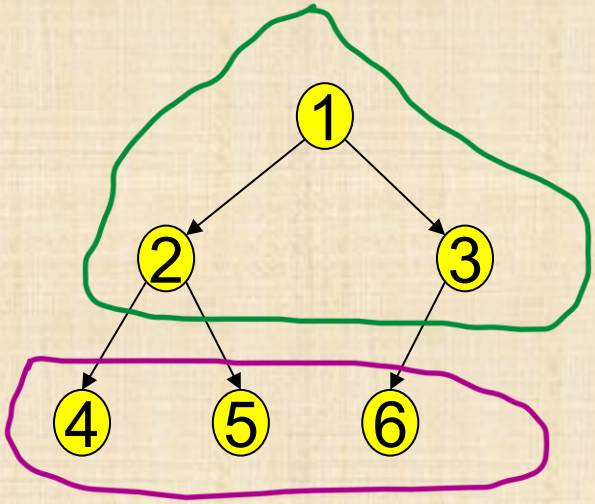
- Organize $N$ nodes in an almost complete binary tree.
- Denote $d$ the depth of the tree organized.
- There are $\sum_{i=0}^{d-1} 2^i = 2^d - 1$ nodes up to depth $d-1$
- And between 1 and $2^{\mathrm{d}}$ nodes at depth $d$
- Thus: If $2^{\mathrm{d}} - 1 < 2^d \leq N \leq 2^d - 1 + 2^d < 2^{d+1}$

$$d \leq \log_2 N < d + 1$$

$$\log_2 N - 1 < d$$

$$\log_2 N - 1 < d \leq \log_2 N$$

- $d$ is an integer, so $d = \lfloor \log_2 N \rfloor$



# Binary Trees

## Is a Tree an ADT
## or
## Logical Data-Structure
## or
## Actual Implementation
## ?

# Binary Trees

**Is a Tree an ADT**

**or**

**Logical Data-Structure**

**or**

**Actual Implementation**

**?**

# Binary Heap

An almost complete binary tree satisfying the heap property:

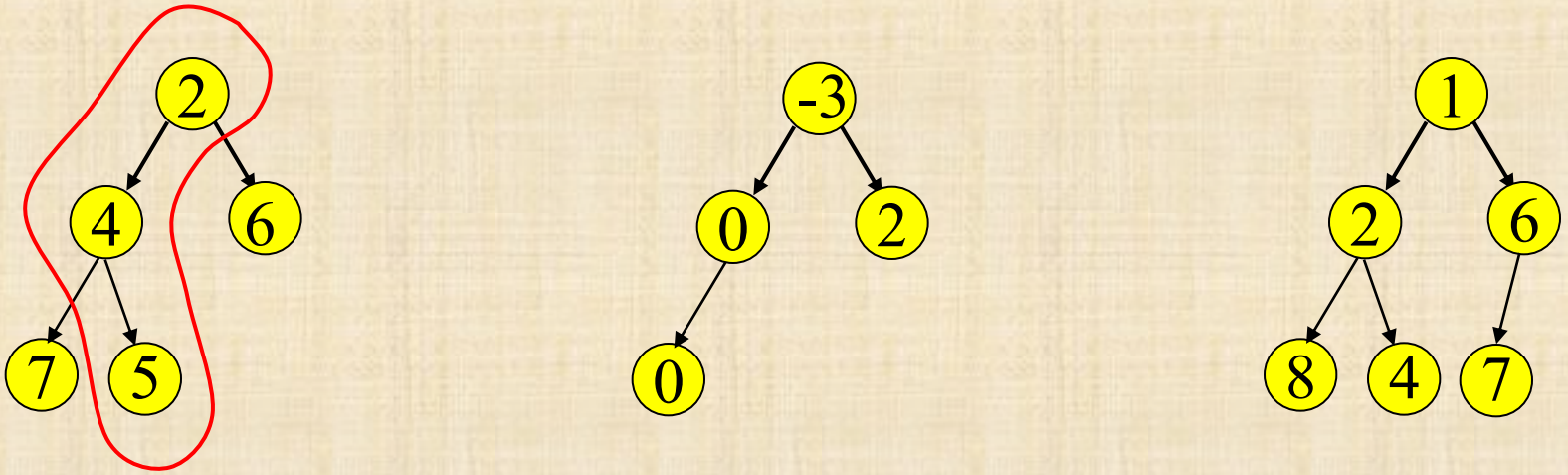
max heap: no node is smaller than its children

or

min heap: no node is larger than its children

# Heap order property

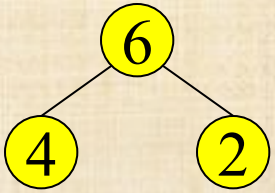
- A heap provides limited ordering guaranties
- The values along each path are sorted, but subtrees are not necessarily sorted relative to each other
  - A binary heap is NOT a binary search tree (to be learnt)



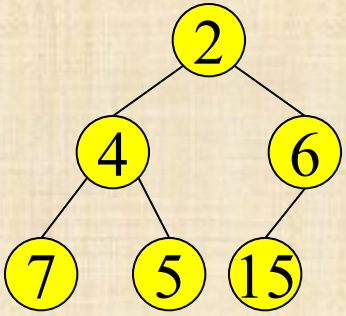

These are all valid binary heaps (minimum)

# Examples

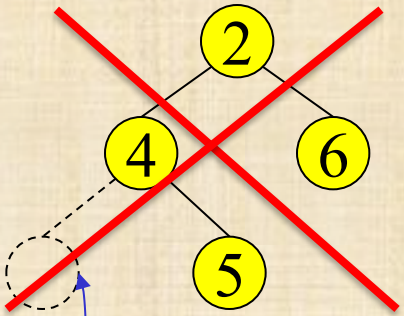
Good



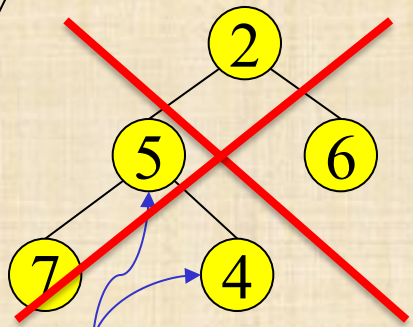

complete tree,
heap order is "max"




almost complete tree,
heap order is "min"




not almost complete

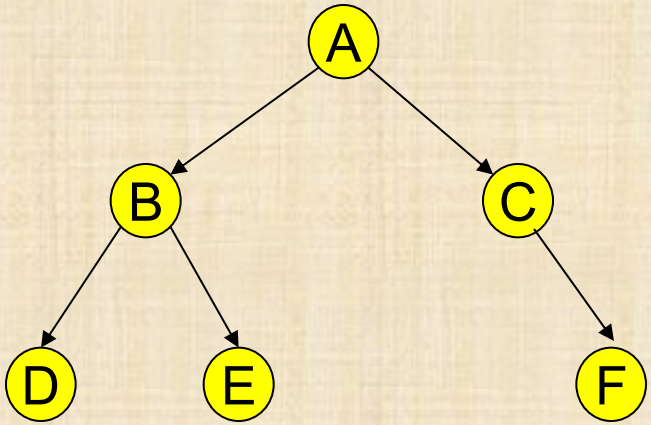


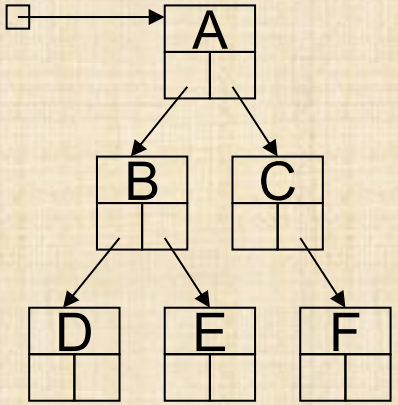

Bad

almost complete tree, but both min and max
heap order is broken

# Implementations of Binary Trees

- Array
- Pointers
  - A structure with value and two pointers, (left child & right child)



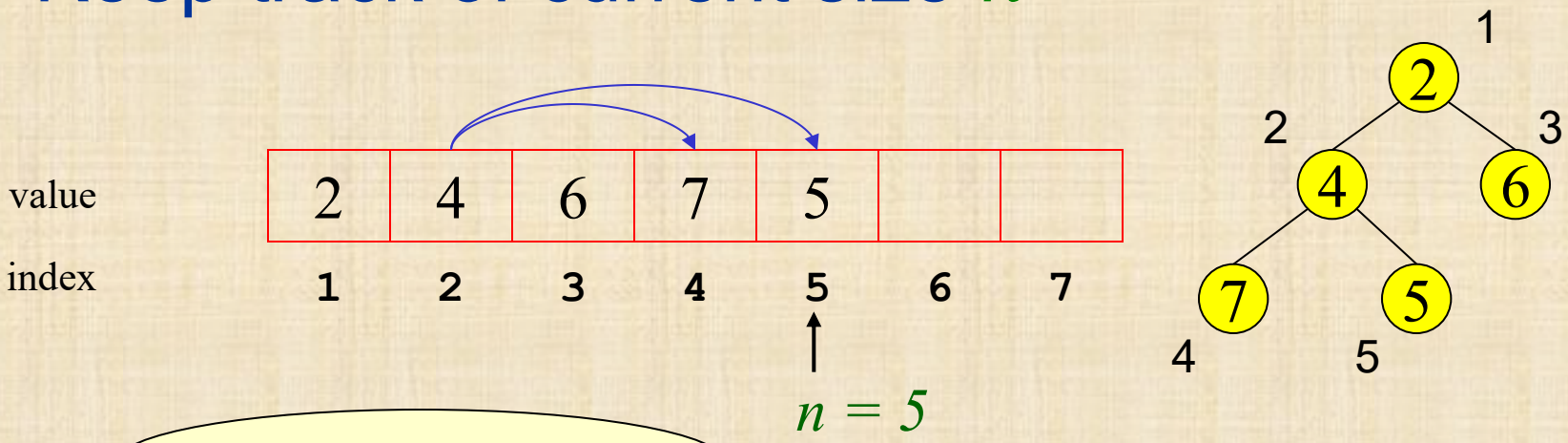
# Binary Trees - Reminder

## A Tree is a

## Logical Data-Structure

# Heap: Array Implementation

- Root node = $A[1]$
- Children of $A[i]$ are in $A[2i]$, $A[2i + 1]$
- Parent of $A[i]$ is in $A[\lfloor i/2 \rfloor]$
- Keep track of current size $n$



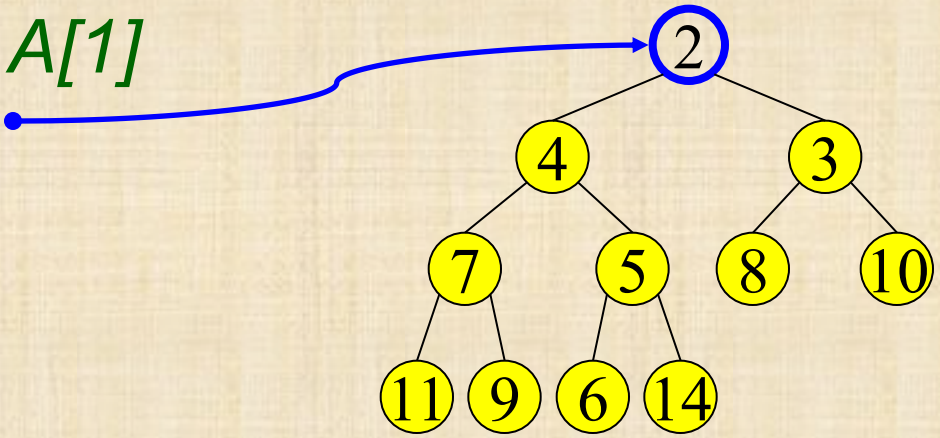
# Priority Queue using a heap

- Goal:
  Use a heap to implement the priority queue operations:

  - Insert(x,k): insert element x with key k
  - FindMin(): return element with minimum key
  - DeleteMin(): delete element with minimum key and return it
  - DecreaseKey(h,k): Decrease to k the key of an existing element in the Queue referred to by the handle h

# FindMin and DeleteMin

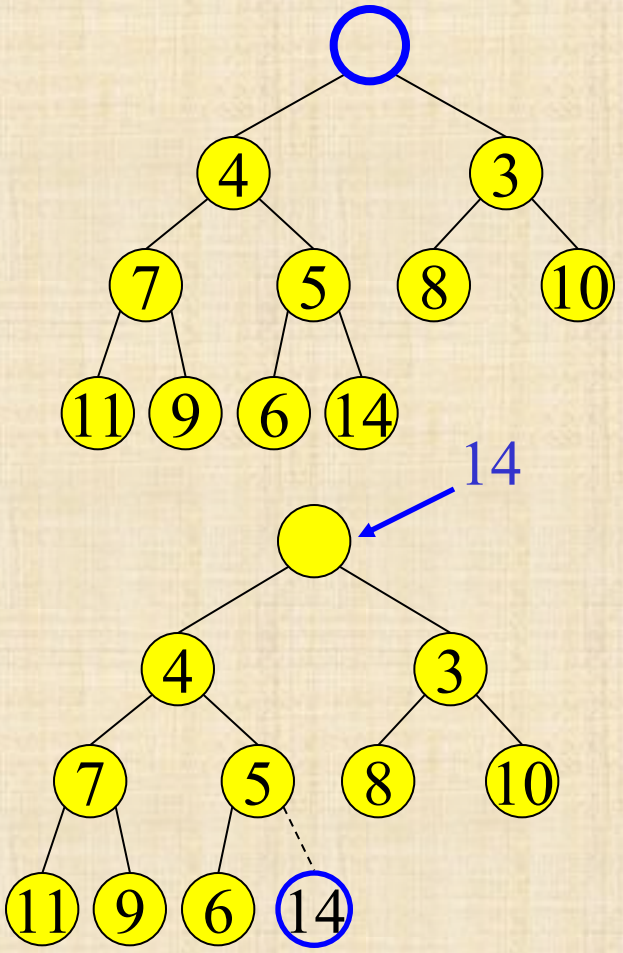
- *FindMin:* Easy!
  - Return root value *A[1]*
  - Run time = ?




- *DeleteMin:*
  - Delete (and return) value at root node
    - How can we delete?

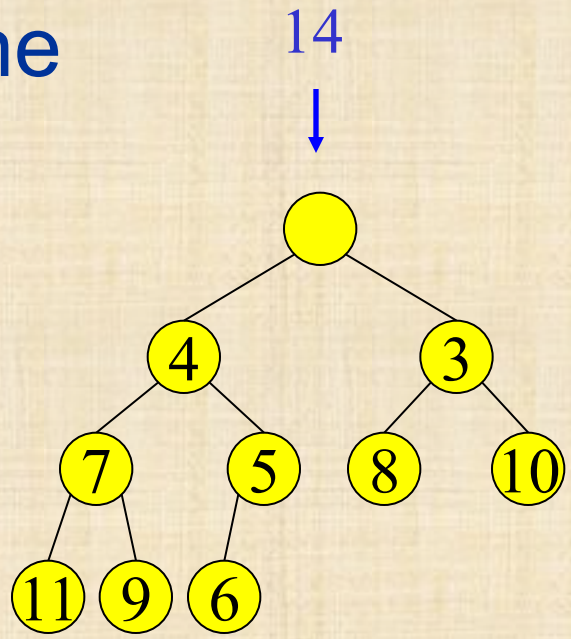
# Maintain the Structure

- Remove a value from the root *A[1]*
- The tree will have one less node and must still be almost complete
- Replace the root with the "last element" , A[n]. any problem?

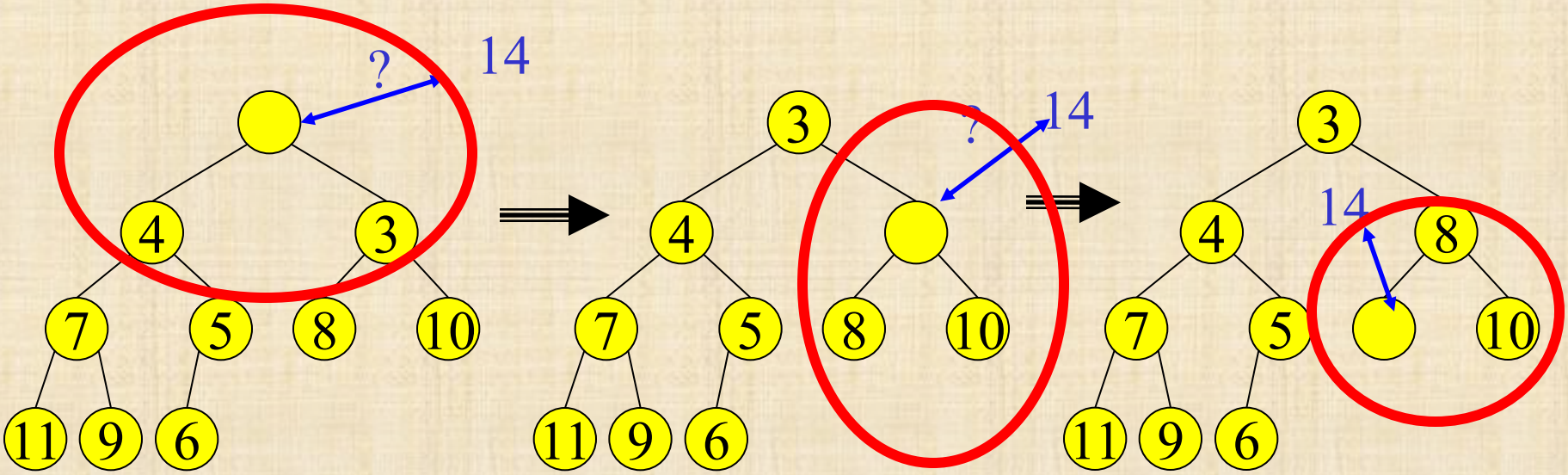


# Maintain the Heap Property

- Move “last element” down as long as long as it violates the heap property



# Percolate Down




- Keep comparing x with children *A[2i]* and *A[2i + 1]*
- Swap locations with the smaller child and call recursively with the sub tree that x is its (new) root
- Terminate if both children are greater than x or reached a leaf node

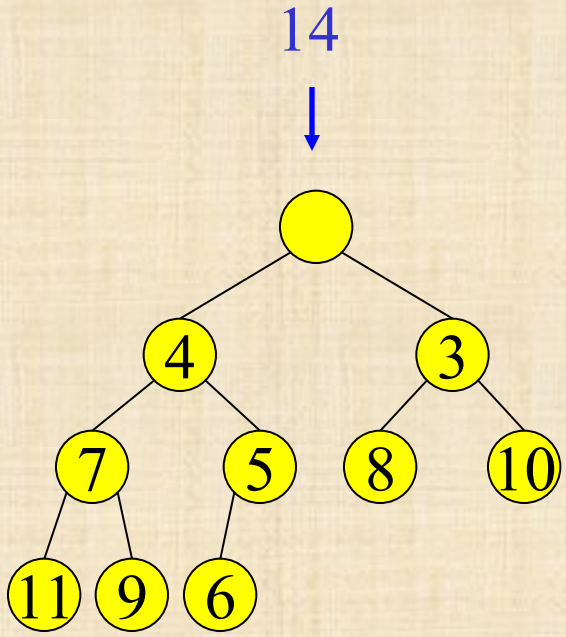
# Recursive Percolate Down

- *We define the function:*

  *PercDown*(i, x, n, A)

- *In our example we call:*

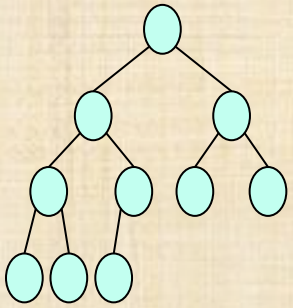
  *PercDown*(1, 14, 10, A)




i: is the index we consider
x: the value to be inserted
n: the number of elements in the heap
A: array that stores the heap

# Percolate Down

```
PercDown(i, x, n, A)
Case:
        2i > n : A[i] ← x;                          % A leaf
        2i = n : if (A[2i] < x) then                % A single child
                             A[i] ← A[2i];
                             A[2i] ← x;
                 else        A[i] ← x;
        2i < n : if (A[2i] < A[2i+1]) then          % two children
                             j ← 2i;
                 else        j ← 2i+1;
                 if (A[j] < x) then
                             A[i] ← A[j];
                             PercDown(j,x,n,A);
                 else        A[i] ← x;
```



How many steps?

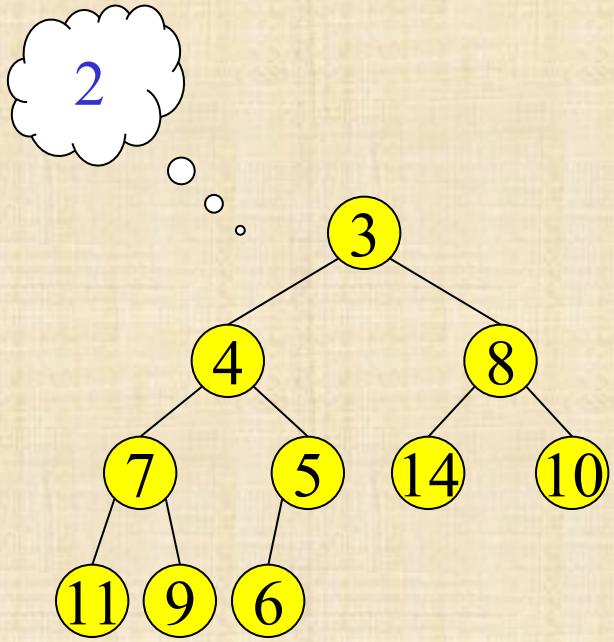
i: is the index we consider
x: the value to be inserted
n: the number of elements in the heap
A: array that stores the heap

# DeleteMin: Run Time Analysis

- A heap is an almost complete binary tree (all levels but the last are full)
- Depth of an almost complete binary tree with *n* nodes?
  - d = $\lfloor \log_2(n) \rfloor$
- O(1) operations at each level of *PercDown*(1, x, n, A)
- Run time of *PercDown*, and hence also of *DeleteMin* is $O(\log_2 n)$

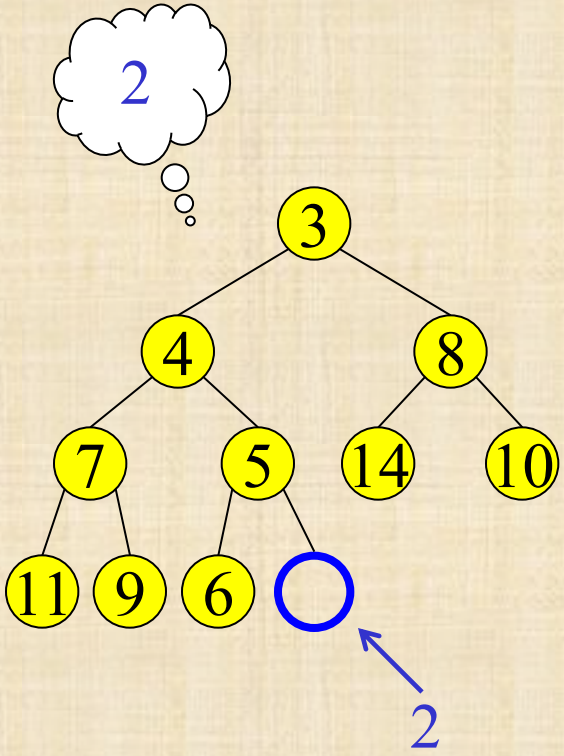
# Insert

- Add a value to the tree
- Structure and heap order properties must still be correct when we are done



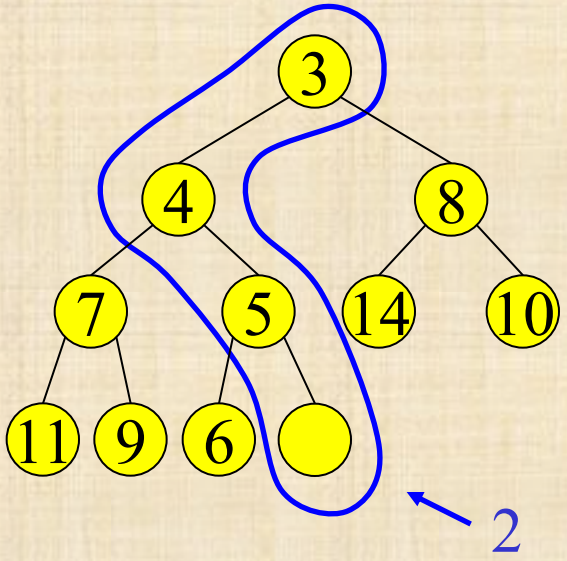
# Maintain the Structure Property

- A valid place for a new node is at the end of the array
- The new value goes to A[n+1] (and *n* is increased)
- Any problem?

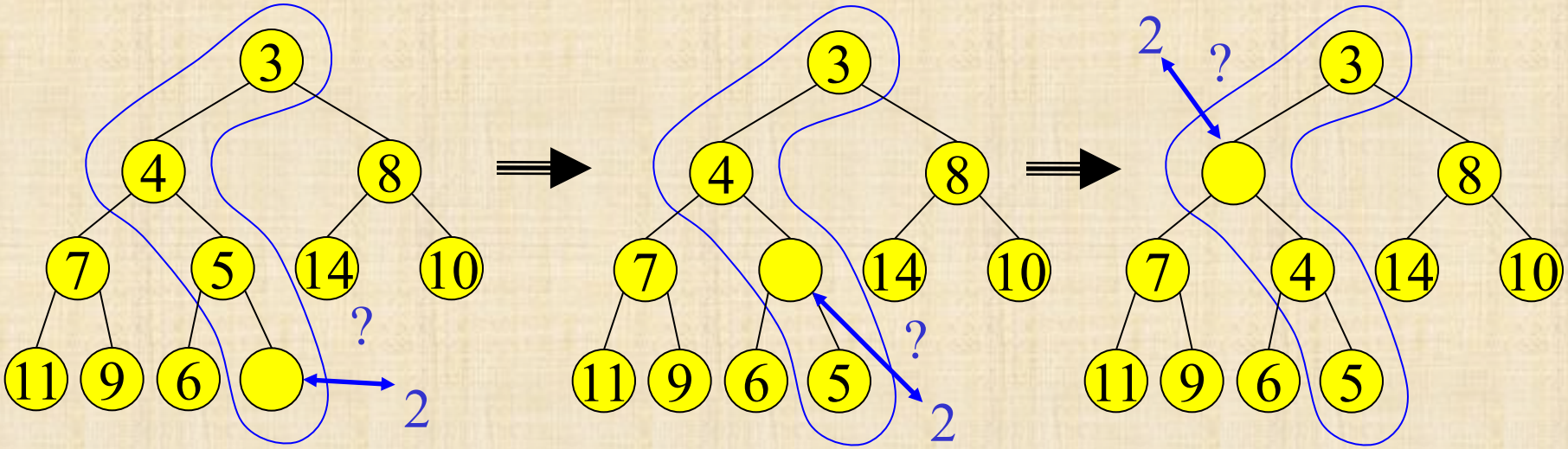


# Maintain the Heap Property

- By *Percolate-Up* which percolates new entry to correct spot

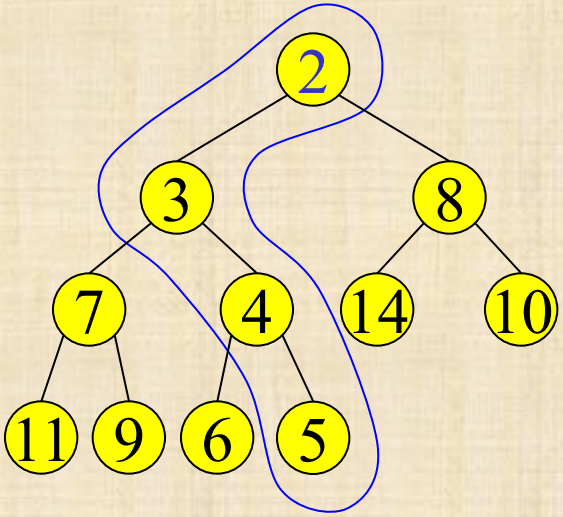


# Percolate Up



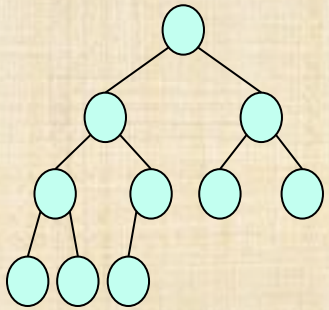

- Start at last node and keep comparing with parent A[⌊i/2⌋]
- If parent larger than x, copy parent down and go up one level
- Done if parent ≤ x or reached top node A[1]

# Insert: Done



# PercUp implementation

```
PercUp(i, x, A)
p = ⌊i/2⌋
if i = 1 then
        A[1] ← x
elseif A[p] < x then
        A[i] ← x;
else
    A[i] ← A[p];
    PercUp(p, x, A);
```



How many steps?

i: is the index we consider
x: the value to be inserted
A: array that stores the heap
p: index of the parent of *i*

# Binary Heap Analysis

- Correctness: Omitted
- Space:
  - An array of size $N$, plus a variable to store the current size $n$
- Time
  - FindMin: $O(1)$
  - DeleteMin and Insert: $O(\log n)$
  - What about DecreaseKey??

# Binary Heap Analysis

- Correctness: Omitted
- Space:
  - An array of size $N$, plus a variable to store the current size $n$
- Time
  - FindMin: $O(1)$
  - DeleteMin and Insert: $O(\log n)$
  - What about DecreaseKey:
    - Change the key
    - PercUp to maintain the heap property

# Binary Heap Analysis

- Correctness: Omitted
- Space:
  - An array of size $N$, plus a variable to store the current size $n$
- Time
  - FindMin: $O(1)$
  - DeleteMin and Insert: $O(\log n)$
  - DecreaseKey: $O(\log n)$

# Binary Heap Analysis

- Correctness: Omitted
- Space:
  - An array of size $N$, plus a variable to store the current size $n$
- Time
  - FindMin: $O(1)$
  - DeleteMin and Insert: $O(\log n)$
  - DecreaseKey: $O(\log n)$
  - What about Building a Heap from an Array with $n$ elements?

# Naïve Build Heap

- Insert all elements of A into a new Heap array B one by one.
- Copy the Heap array back into A

```
NaïveBuildHeap(A,n)
Allocate array B of size n
for i = 1 to n
        Insert(B,i,A[i])
for i = 1 to n
        A[i]=B[i]
```

# Naïve Build Heap Complexity

- *NaïveBuildHeap* impelementation
  - $n$ calls to **`Insert`**. Each invokes **`PercUp`**.
  - Time for **`PercUp`** is proportional to the depth of the current tree (not all elements were inserted yet)
  - After $\frac{n}{2}$ insertions, depth of tree is $\log \frac{n}{2}$
  - Time for the last $\frac{n}{2}$ insertions is $\geq \frac{n}{2} \log \frac{n}{2}$

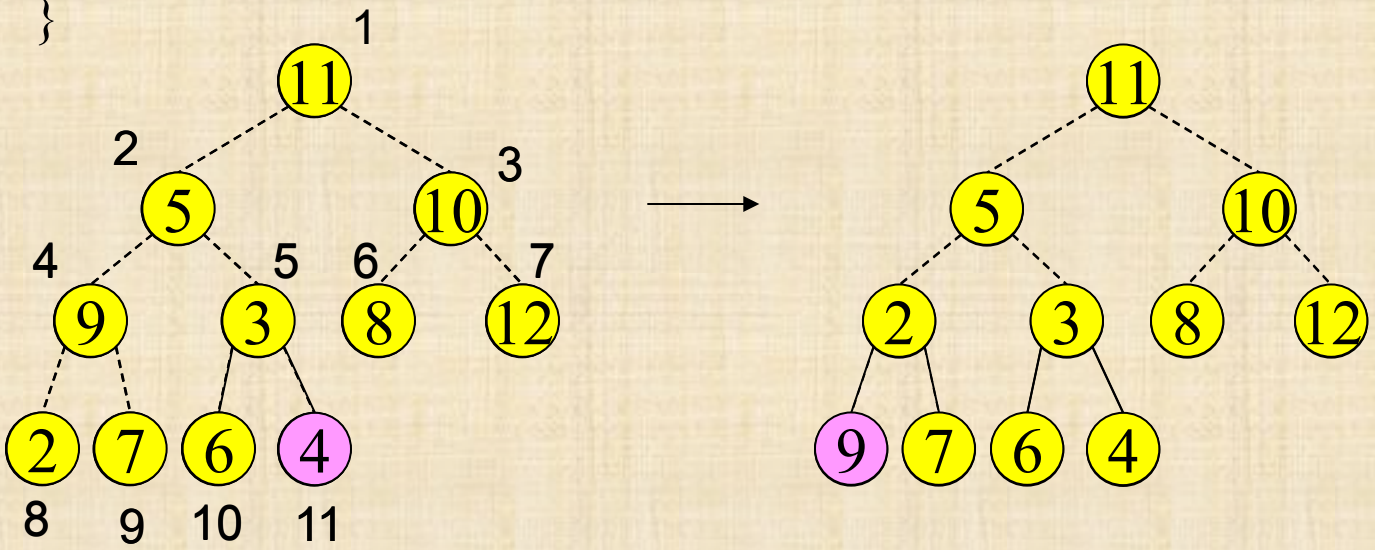
- We can do better: $O(n)$

# Efficient Build Heap bottom up

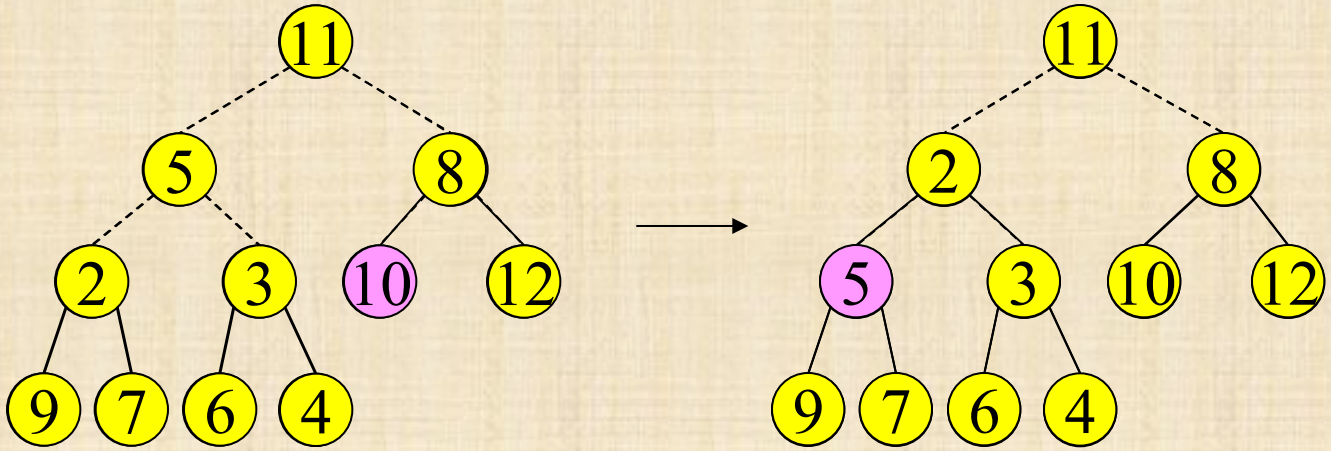
```
BuildHeap (A) {
  n ← length(A)
  for i = ⌊n/2⌋ down to 1
      PercDown(i, A[i], n, A)
}
```

A: array that stores the heap
n: number of elements in the heap
i: is the index we consider

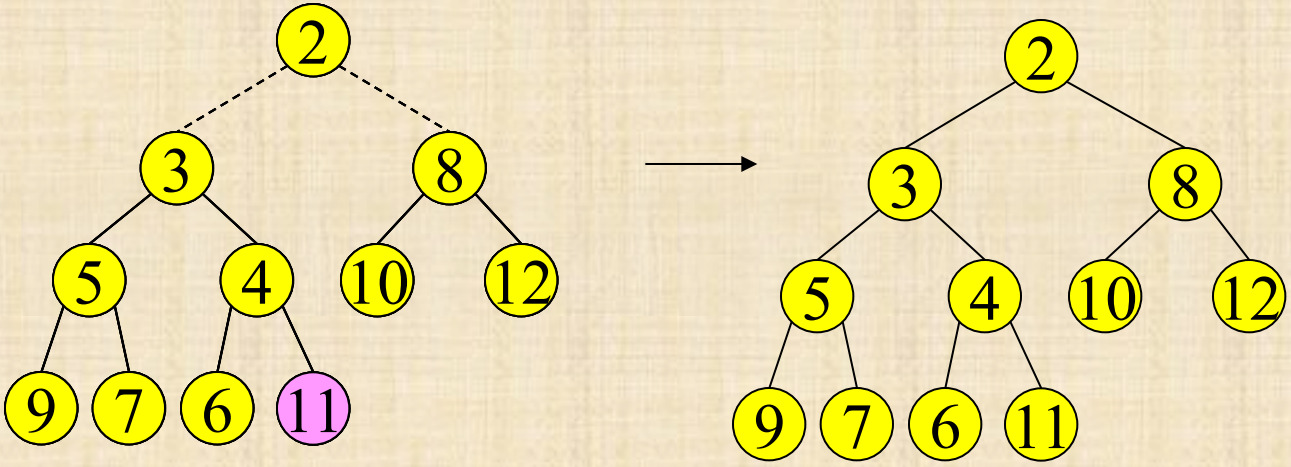
n=11



# Build Heap



# Build Heap



# Correctness

- We prove by reverse induction (we start with *n* and work our way towards *1*) that after handling element A[$i$], A[$i \ldots n$] satisfies the heap property, that is, no element in A[$i \ldots n$] is larger than its children.
- Base: $i = \left\lfloor \frac{n}{2} \right\rfloor + 1$, A[$i \ldots n$] contains only leaves, so A[$i \ldots n$] already satisfies the heap property
- Induct. hypothesis: After handling element A[$i + 1$], A[$(i + 1) \ldots n$] satisfies the heap property
- Step: For $i \leq \left\lfloor \frac{n}{2} \right\rfloor$, by the inductive hypothesis, the only element in A[$i \ldots n$] that might not satisfy the heap property is A[$i$] itself. Hence, after *PercDown*(i,A[$i$],$n$,A), the heap property is satisfied for A[$i \ldots n$]

# Analysis of Build Heap

- Efficient Build Heap :
  - Insert all keys into an array in arbitrary order,
    then turn it to a heap:
  - for i = $\lfloor n/2 \rfloor$ downto 1

    *PercDown*(i, A[i], n, A)

- How long does it take?

# Analysis of Build Heap

- Recall: If a node has height *h*, then percolating down the node takes *O(h)* time

- Assume $\mathrm{N} = 2^{k+1} - 1$ (a complete tree of height $k$)
  - Level 0 (root): $k$ steps for 1 item
  - Level 1: $k - 1$ steps for 2 items
  - Level 2: $k - 2$ steps for 4 items
  - Level $i$ : $k - i$ steps for $2^i$ items
  - Level $k$: 0 steps for $2^k$ items

- Summing over all levels:

$$\sum_{i=0}^{k} (k-i)2^i$$

# Analysis of Build Heap

$$\sum_{i=0}^{k}(k-i)2^i =$$

# Analysis of Build Heap

$$\sum_{i=0}^{k}(k-i)2^i = \sum_{i=0}^{k} i2^{k-i} =$$

# Analysis of Build Heap

$$\sum_{i=0}^{k}(k-i)2^i = \sum_{i=0}^{k} i2^{k-i} = 2^k \sum_{i=0}^{k} i2^{-i} \leq$$

# Analysis of Build Heap

$$\sum_{i=0}^{k}(k-i)2^i = \sum_{i=0}^{k} i2^{k-i} = 2^k \sum_{i=0}^{k} i2^{-i} \leq N \sum_{i=0}^{k} i2^{-i} \leq$$

# Analysis of Build Heap

$$\sum_{i=0}^{k}(k-i)2^i = \sum_{i=0}^{k} i2^{k-i} = 2^k \sum_{i=0}^{k} i2^{-i} \leq N \sum_{i=0}^{k} i2^{-i} \leq$$

$$\leq N \sum_{i=0}^{k} \left(\frac{3}{2}\right)^i 2^{-i} =$$

# Analysis of Build Heap

$$\sum_{i=0}^{k}(k-i)2^i = \sum_{i=0}^{k} i2^{k-i} = 2^k \sum_{i=0}^{k} i2^{-i} \leq N \sum_{i=0}^{k} i2^{-i} \leq$$

$$\leq N \sum_{i=0}^{k} \left(\frac{3}{2}\right)^i 2^{-i} = N \sum_{i=0}^{k} \left(\frac{3}{4}\right)^i \leq$$

# Analysis of Build Heap

$$\sum_{i=0}^{k}(k-i)2^i = \sum_{i=0}^{k} i2^{k-i} = 2^k \sum_{i=0}^{k} i2^{-i} \leq N \sum_{i=0}^{k} i2^{-i} \leq$$

$$\leq N \sum_{i=0}^{k} \left(\frac{3}{2}\right)^i 2^{-i} = N \sum_{i=0}^{k} \left(\frac{3}{4}\right)^i \leq 4 \cdot N =$$

# Analysis of Build Heap

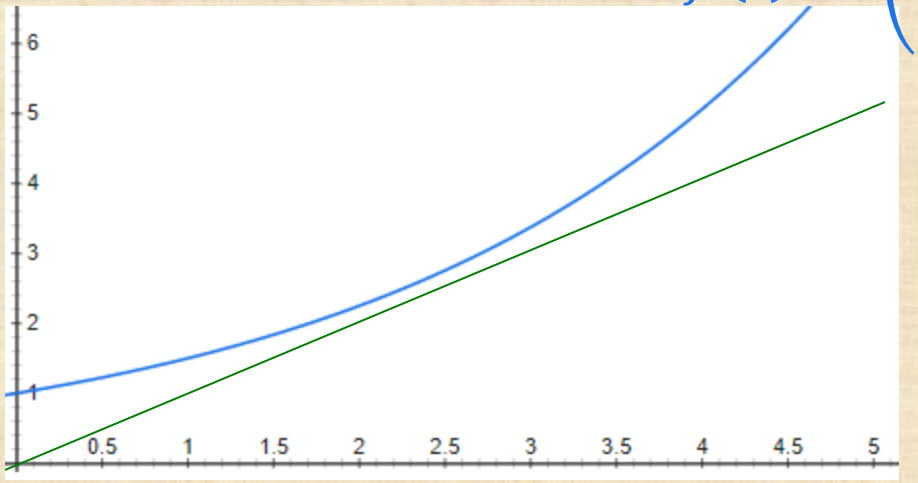
$$\sum_{i=0}^{k}(k-i)2^i = \sum_{i=0}^{k} i2^{k-i} = 2^k \sum_{i=0}^{k} i2^{-i} \leq N \sum_{i=0}^{k} i2^{-i} \leq$$

$$\leq N \sum_{i=0}^{k} \left(\frac{3}{2}\right)^i 2^{-i} = N \sum_{i=0}^{k} \left(\frac{3}{4}\right)^i \leq 4 \cdot N = O(N)$$

# Analysis of Build Heap

- for every $i \geq 0$, it holds that $i \leq \left(\frac{3}{2}\right)^i$

$$f(i) = \left(\frac{3}{2}\right)^i$$




$$f(i) = i$$

# Summary

- Linear order: sequence, linked list, queue, stack
- hierarchical order: trees
- Heap: priority queue
  - FindMin: $O(1)$
  - DeleteMin and Insert: $O(\log n)$
  - DecreaseKey: $O(\log n)$
  - BuildingHeap: $O(n)$

Trees

Binary Heaps

# Next Lecture

- Asymptotic complexity definition:

$$O(n), \Omega(n), \Theta(n)$$

- Best case, worst case, average case, amortized analysis